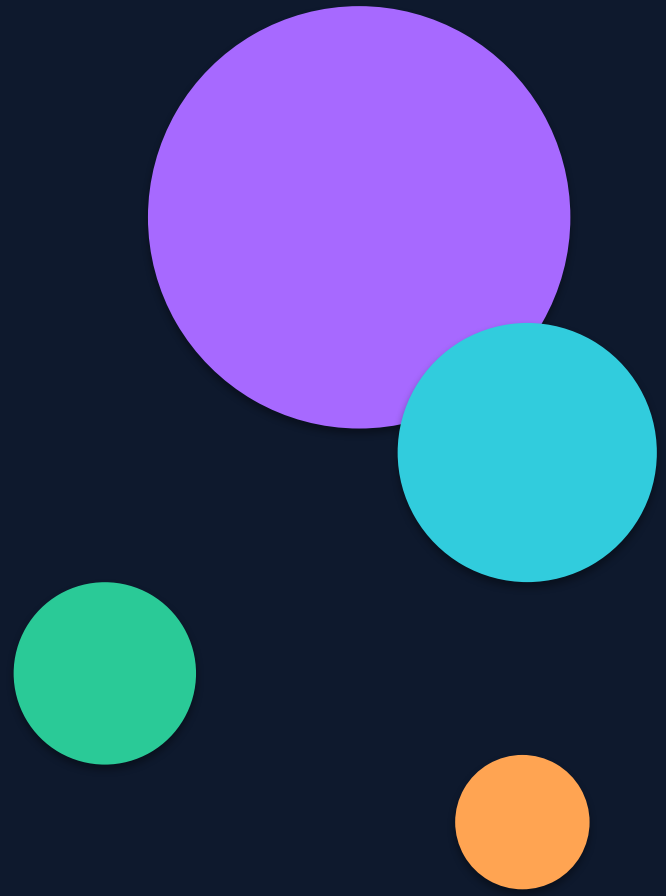
Introduction to Data Science & Python

Detailed compact notes with concepts, methods, examples and practical reminders.



6 topic slides + cover = 7 slides

Data Science: Meaning and Scope

Key concepts • practical context • quick revision

- Data science extracts useful knowledge from data by combining programming, statistics, domain expertise and communication.
- Main goals: describe what happened, diagnose why it happened, predict what may happen and recommend what to do.
- Data forms: structured (tables), semi-structured (JSON/XML) and unstructured (text, images, audio and video).
- Common roles: data analyst, data scientist, data engineer, ML engineer and business intelligence analyst.
- Typical outputs include reports, dashboards, forecasts, classifiers and recommendation systems.

EXAMPLE / REMEMBER

Example: an online store studies orders to understand popular products and forecast future demand.

End-to-End Data Science Lifecycle

Key concepts • practical context • quick revision

- 1. Define the problem and success metric; clarify who will use the result.
- 2. Collect relevant data and document its source, meaning, date and limitations.
- 3. Clean and transform data; check missing values, duplicates, types and errors.
- 4. Explore data using summaries and visualizations to identify patterns and questions.
- 5. Build a baseline, train suitable models and tune them using training data.
- 6. Evaluate on unseen data, communicate limitations, deploy and monitor performance.
- The lifecycle is iterative: poor results may require better data, features or problem definition.

EXAMPLE / REMEMBER

A model is useful only when it answers the original question reliably.

Python Fundamentals

Key concepts • practical context • quick revision

- Variables are names that refer to values; Python uses dynamic typing.
- Core types: int (whole numbers), float (decimals), str (text), bool (True/False).
- Operators perform arithmetic, comparison and logical operations.
- if/elif/else selects a path; for and while repeat operations.
- Functions use def to package reusable logic; parameters accept inputs and return sends back a result.
- Indentation defines blocks. Meaningful names and small functions improve readability.

EXAMPLE / REMEMBER

```
Example:
marks = [72, 85, 91, 68]
average = sum(marks) /
len(marks)
print(average)  # 79.0
```

Python Collections and Files

Key concepts • practical context • quick revision

- List: ordered and mutable; useful for sequences of records or values.
- Tuple: ordered and immutable; useful for fixed groups of values.
- Dictionary: key-value mapping; useful for representing a record or lookup table.
- Set: unique elements; useful for membership tests and removing duplicates.
- Files can be read from CSV, JSON, text or Excel using appropriate libraries.
- Use exceptions and validation to handle missing files, malformed rows and invalid values.

EXAMPLE / REMEMBER

```
Dictionary example:
student = {'name':'Asha',
'marks':85}
print(student['marks'])
```

NumPy and Pandas

Key concepts • practical context • quick revision

- NumPy provides efficient multidimensional arrays and vectorized mathematical operations.
- Pandas Series represents one labeled column; DataFrame represents a labeled table.
- Read data with read_csv/read_excel; inspect with head(), shape, info() and describe().
- Select columns, filter rows, sort values, group records and combine tables with joins/merge.
- Missing values can be detected with isna(); duplicates with duplicated() or drop_duplicates().
- Prefer vectorized operations over slow manual loops for large tabular calculations.

EXAMPLE / REMEMBER

```
Starter:
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head())
print(df.isna().sum())
```

Visualization and Scientific Tools

Key concepts • practical context • quick revision

- Matplotlib provides flexible plots; Seaborn simplifies statistical visualizations.
- Common charts: bar for categories, histogram for distributions, scatter for relationships and line for time.
- Scikit-learn supplies preprocessing, estimators, train/test splitting and evaluation metrics.
- Jupyter notebooks combine explanatory text, executable code, output and visualizations.
- Choose a tool based on the question, dataset size, readability and reproducibility.
- Save the steps and package versions so another person can reproduce the analysis.

EXAMPLE / REMEMBER

Remember: a chart should answer a question, not merely decorate a report.